

Nome: \_\_\_\_\_

- As questões que envolvem implementação devem ser resolvidas utilizando a linguagem C/C++.
- Utilize as folhas de resposta distribuídas junto com a prova para responder às questões.
- Em cada folha, escreva seu nome completo no canto superior esquerdo. Respostas em folhas sem identificação não serão corrigidas.
- As respostas podem ser escritas a lápis ou caneta. Certifique-se de que sua escrita está legível.
- Indique claramente o número da questão que está respondendo e, quando necessário, detalhe os passos utilizados para chegar à solução.
- Ao final da prova, as folhas de resposta devem ser grampeadas junto à prova.

---

**Questão 1** (2.5 pontos) Uma árvore binária de busca (BST) é representada pela seguinte estrutura:

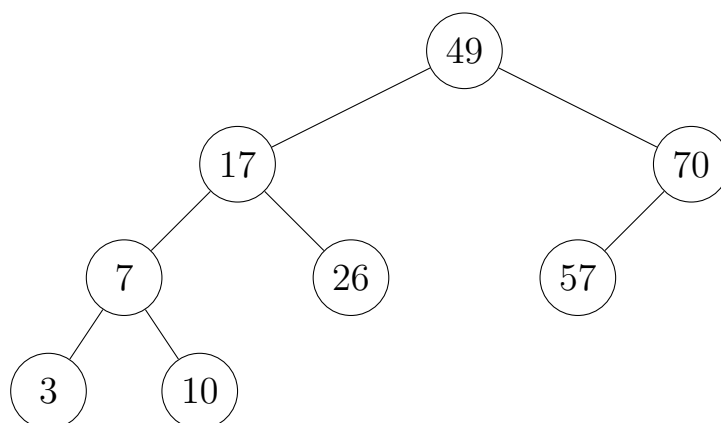
```
struct Node {  
    int key;        // Valor armazenado no nó  
    Node* left;     // Ponteiro para o filho à esquerda  
    Node* right;    // Ponteiro para o filho à direita  
    Node* parent;   // Ponteiro para o pai  
};
```

Suponha que desejamos encontrar o  $k$ -ésimo menor elemento armazenado nessa árvore.

- Implemente a função `int findKthSmallest(Node* root, int k)` que retorne o valor do  $k$ -ésimo menor elemento da BST.
- Qual é a complexidade da sua solução no pior caso? Justifique.
- É possível adicionar alguma informação à estrutura `Node` para reduzir o custo computacional dessa operação? Explique qual informação seria adicionada, como ela seria mantida e qual seria o ganho de desempenho.

---

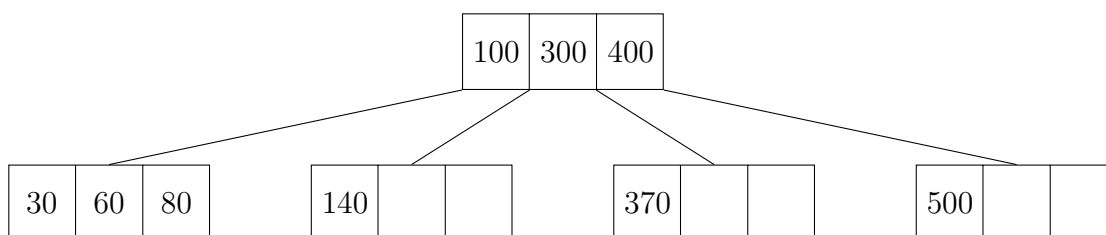
**Questão 2** (2.0 pontos) Suponha a árvore AVL mostrada a seguir.



Como ficará a árvore se realizarmos cada uma das operações abaixo? Desenhe a nova árvore para cada caso. **Atenção:** cada uma das operações ocorre na árvore original acima.

- (a) inserção com chave 65
- (b) inserção com chave 13
- (c) remoção com chave 26
- (d) remoção da chave 70

**Questão 3** (1.5 pontos) Considere a árvore 2-3-4 abaixo, correspondente a uma árvore B com grau mínimo  $t = 2$ :



Como ficará a árvore se realizarmos cada uma das operações a seguir, seguindo os algoritmos para árvores B vistos em aula? Desenhe a nova árvore para cada caso. **Atenção:** Cada uma das operações ocorre a árvore original acima.

- (a) inserção com chave 70
- (b) remoção com chave 140
- (c) remoção com chave 370

---

**Questão 4** (2.0 pontos) Considere a representação a seguir para os nós de uma árvore rubro-negra:

```
struct Node {
    int key;        // Valor armazenado no nó
    int isRed;      // Cor do nó: 1 = vermelho, 0 = preto
    Node* left;     // Ponteiro para o filho à esquerda
    Node* right;    // Ponteiro para o filho à direita
    Node* parent;   // Ponteiro para o pai
};
```

Uma árvore rubro-negra é uma árvore binária que deve satisfazer as seguintes propriedades:

1. Todo nó é vermelho ou preto;
2. A raiz e os nós folha (NIL) são sempre pretos;
3. Se um nó é vermelho, então seus filhos são pretos (não há dois vermelhos consecutivos);
4. Todos os caminhos de um nó até qualquer folha contêm o mesmo número de nós pretos.

Implemente as funções abaixo que verificam as propriedades (3) e (4) descritas acima, assumindo que os nós NIL são representados explicitamente na árvore como nós com ponteiros definidos e cor preta.

- (a) Implemente a função *int checkRedNodeChildren(Node\* root, Node\* NIL)* que retorna 1 se todos os nós vermelhos têm filhos pretos, e 0 caso contrário.
- (b) Implemente a função *int checkBlackHeight(Node\* root, Node\* NIL)* que retorna 1 se todos os caminhos da raiz até os nós folha NIL contêm o mesmo número de nós pretos, e 0 caso contrário.

**Questão 5** (2.0 pontos)

Um sistema de recomendação de uma plataforma de streaming (como vídeos, músicas ou notícias) mantém uma lista de conteúdos candidatos a serem exibidos para um usuário. Cada item recebe uma pontuação de relevância, que pode mudar dinamicamente conforme o comportamento do usuário (ex: cliques, tempo de consumo, avaliações). O sistema precisa garantir que, a qualquer momento, os conteúdos mais relevantes sejam mostrados primeiro. Para isso, as seguintes operações devem ser suportadas com eficiência:

- **Inserir** um novo conteúdo com uma pontuação inicial de relevância;
- **Remover** um conteúdo que não deve mais ser exibido;
- **Atualizar** a pontuação de relevância de um conteúdo existente;
- **Recuperar e remover os K conteúdos mais relevantes** no momento atual.

Considere que cada conteúdo é identificado por um identificador único.

---

Com base nesse cenário, responda:

- (a) Dentre as estruturas de dados apresentadas em aula, qual (ou quais) você usaria para implementar esse sistema? Justifique sua escolha considerando os requisitos de desempenho.
- (b) Descreva, de forma geral, como cada uma das quatro operações poderia ser implementada com as estruturas escolhidas.
- (c) Indique e justifique a complexidade de tempo (no pior caso) de cada operação.